40. Design Key-Value Store

Key Terms:

Term	Full Name	What it does
LSM Tree	Log-Structured Merge Tree	Write-optimized storage with sequential writes
MemTable	In-memory table	Sorted buffer for recent writes
SSTable	Sorted String Table	Immutable on-disk sorted file
WAL	Write-Ahead Log	Durability log before memory write
Compaction	Merge process	Combines SSTables, removes tombstones
Tombstone	Delete marker	Marks deleted keys (removed during compaction)
Quorum	Majority agreement	W+R>N ensures consistency

Problem Statement

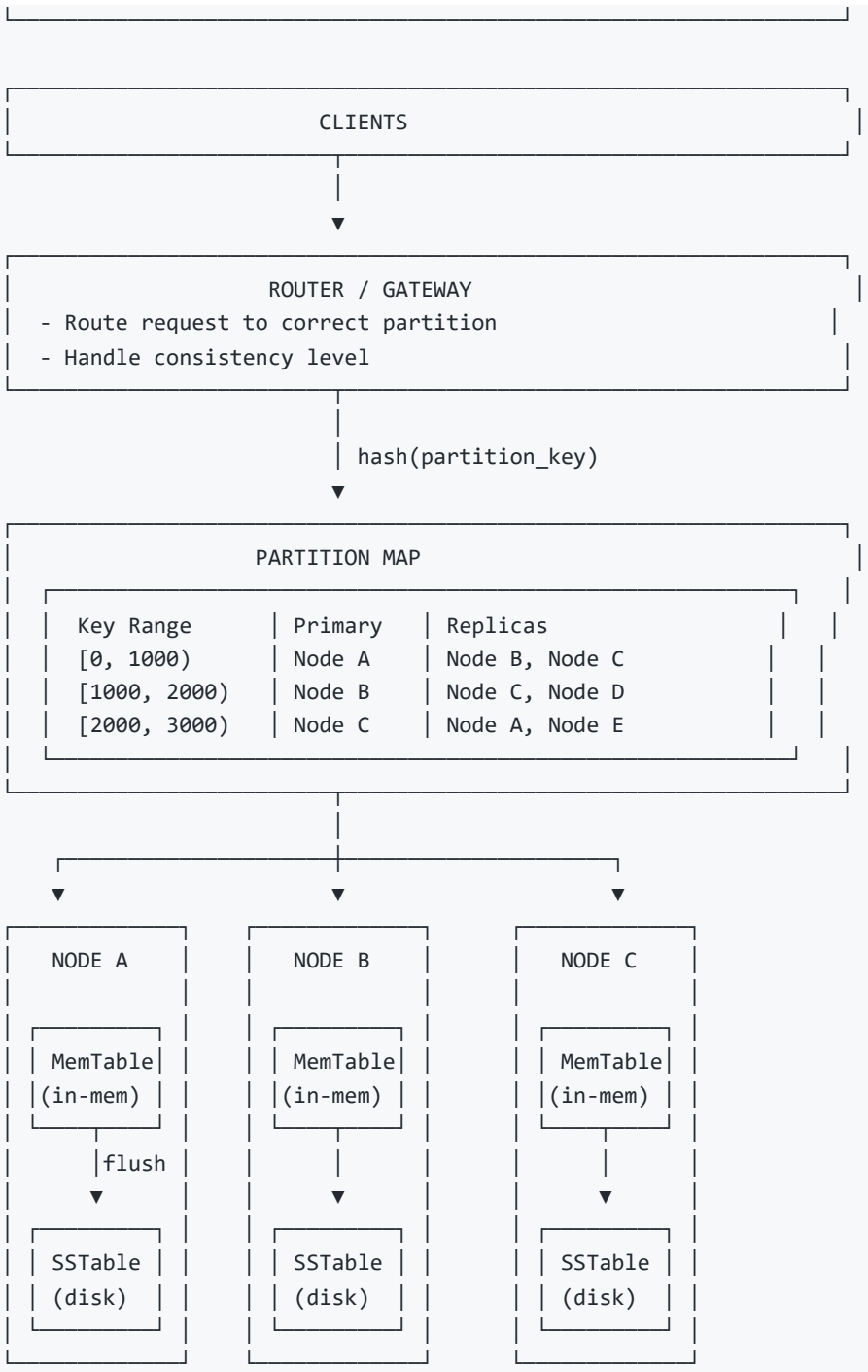
Design a distributed key-value store like Cosmos DB or DynamoDB that provides high availability, scalability, and tunable consistency.

Clarifying Questions to Ask

1. **Scale**: Data size? QPS? (Petabytes, millions QPS?)
2. **Consistency**: Tunable or fixed?
3. **Availability**: Multi-region active-active?
4. **Data model**: Just KV or support secondary indexes?
5. **Transactions**: Single-key or multi-key transactions?
6. **Latency**: Single-digit millisecond required?
7. **Durability**: What's acceptable data loss?

High-Level Design

```
┌─────────────────────────────────────────────────────────────────┐
│                    KEY-VALUE STORE ARCHITECTURE                 │
```



API Design

Core Key-Value APIs

Method	Endpoint	Description
`GET`	`/api/v1/kv/{key}`	Get value by key
`PUT`	`/api/v1/kv/{key}`	Put key-value pair

`DELETE`	`/api/v1/kv/{key}`	Delete a key
`GET`	`/api/v1/kv?prefix={prefix}&limit=100`	Range scan by prefix

Put Request with Options

```
PUT /api/v1/kv/user:123
{
  "value": "eyJuYW1lIjoiSm9obiJ9",  // base64 encoded
  "ttl_seconds": 86400,
  "consistency": "quorum",
  "if_version": 5                     // optimistic concurrency
}

Response: 200 OK
{
  "key": "user:123",
  "version": 6,
  "timestamp": 1672531200000
}
```

Batch APIs

Method	Endpoint	Description
`POST`	`/api/v1/kv/batch/get`	Get multiple keys
`POST`	`/api/v1/kv/batch/put`	Put multiple key-value pairs

Admin APIs

Method	Endpoint	Description
`GET`	`/api/v1/cluster/status`	Get cluster health and node info
`GET`	`/api/v1/cluster/partitions`	Get partition map
`POST`	`/api/v1/cluster/rebalance`	Trigger partition rebalancing

Key Components

Component	Purpose	Technology
Router	Direct requests to partitions	Stateless service

Partition Manager	Track partition ownership	Consensus (Paxos/Raft)
Storage Engine	Persist data	LSM tree (RocksDB)
Replication	Data redundancy	Chain or quorum
Compaction	Merge SSTables	Background process

LSM Tree Storage Engine

```
WRITE PATH:
┌─────────────────────────────────────────────────────────────┐
│  1. Write to Write-Ahead Log (WAL) - durability             │
│  2. Write to MemTable (in-memory sorted structure)          │
│  3. When MemTable full → flush to SSTable (disk)            │
│  4. Background compaction merges SSTables                   │
│                                                             │
│  Write: O(1) amortized - always sequential                  │
└─────────────────────────────────────────────────────────────┘

READ PATH:
┌─────────────────────────────────────────────────────────────┐
│  1. Check MemTable                                          │
│  2. Check Bloom filters of SSTables                         │
│  3. Search SSTables (newest to oldest)                      │
│  4. Return first match                                      │
│                                                             │
│  Read: O(log N) with Bloom filter optimization              │
└─────────────────────────────────────────────────────────────┘

COMPACTION:
┌─────────────────────────────────────────────────────────────┐
│  Level 0: Flushed MemTables (may overlap)                   │
│  Level 1+: Merged, non-overlapping ranges                   │
│                                                             │
│  Compaction merges levels, removes tombstones               │
│  Trade-off: Write amplification vs read performance         │
└─────────────────────────────────────────────────────────────┘
```

Consistency Levels

```
TUNABLE CONSISTENCY (like Cosmos DB):
┌─────────────────────────────────────────────────────────────┐
│                                                             │
│  STRONG: Wait for all replicas (highest latency)            │
│    W: Write to all replicas before ACK                      │
│    R: Read from all, return latest                          │
│                                                             │
│  BOUNDED STALENESS: Within K versions or T seconds          │
```

```
│    Guarantees: No reads older than bound                    │
│                                                               │
│  SESSION: Monotonic reads within session                     │
│    Guarantees: Read-your-writes, no going backwards          │
│                                                               │
│  EVENTUAL: Fastest, may return stale                         │
│    W: Write to any replica, async propagate                  │
│    R: Read from any replica                                  │
│                                                               │
└──────────────────────────────────────────────────────────────┘

QUORUM FORMULA:
  N = total replicas
  W = write quorum
  R = read quorum

  Strong consistency if: W + R > N

  Example (N=3): W=2, R=2 → Always see latest write
```

Scalability Considerations

- **Partitioning**: Hash partition key for even distribution
- **Replication**: 3+ replicas across failure domains
- **Auto-scaling**: Split hot partitions automatically
- **Global distribution**: Multi-region with conflict resolution
- **Indexing**: Local secondary indexes per partition

Trade-offs

Decision	Option A	Option B	Recommendation
Storage	B-tree	LSM tree	**LSM** - better write throughput
Consistency	Fixed	Tunable	**Tunable** - flexibility
Partitioning	Range	Hash	**Hash** - even distribution
Replication	Chain	Quorum	**Quorum** - tunable consistency

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference

LSM Tree	Write-optimized storage engine structure	Tech Guide
Cosmos DB	Tunable consistency and global distribution	Tech Guide
Bloom Filter	Efficient SSTable lookup optimization	Tech Guide
Raft Consensus	Partition ownership and replication	Tech Guide

---

How to Explain It Well

> **Opening**: "Key-value stores need high write throughput and tunable consistency. I'll use LSM trees for storage and quorum replication."

> **Key talking points**:
>
> - "LSM tree: writes are always sequential, great throughput"
> - "Bloom filters eliminate unnecessary SSTable reads"
> - "Quorum replication: W+R>N for strong consistency"
> - "Partition by hash for even distribution"
> - "Tunable consistency: strong for critical, eventual for counters"
